# QCM

1

## QCM

**Q1.** Quel fichier contient les chaînes de caractères réutilisables dans Android ?

a) colors.xml

b) strings.xml

c) styles.xml

d) dimens.xml

**Q2.** Où définit-on les menus dans Android ?

a) res/layout/

b) res/menu/

c) res/values/

d) src/java/

2

# QCM

**Q3.** Quelle est la fonction du fichier AndroidManifest.xml ?

a) Gérer la couleur du thème

b) Définir la structure du projet

c) Déclarer les activités et permissions

d) Stocker les données utilisateur

**Q4.** Où sont stockées les ressources dans un projet Android ?

a) Dans le dossier src/

b) Dans le dossier res/

c) Dans le dossier bin/

d) Dans le dossier assets/

3

# QCM

**Q5.** Dans quel sous-dossier place-t-on les fichiers de mise en page ?

a) res/layout/

b) res/drawable/

c) res/values/

d) res/menu/

**Q6.** Quel est le rôle du fichier R.java (ou R.kt) généré automatiquement ?

a) Compiler les layouts XML

b) Créer une référence interne à toutes les ressources du projet

c) Gérer les permissions Android

d) Convertir les images en format binaire

4

## QCM

**Q7**. Le fichier colors.xml permet de :

a) Définir les couleurs utilisées dans l'application

b) Définir la taille des marges

c) Traduire les chaînes de caractères

d) Créer les layouts

**Q8**. Le fichier dimens.xml contient :

a) Les couleurs

b) Les dimensions réutilisables (marges, tailles de texte, etc.)

c) Les textes

d) Les menus

5

## QCM

**Q9**. Quelle ressource est la plus adaptée pour définir une icône personnalisée ?

a) layout

b) drawable

c) Menu

d) values

**Q10**. Comment accède-t-on à une ressource dans le code Kotlin ?

a) Avec son nom directement

b) Avec R.type.nomRessource

c) Avec findViewById()

d) Avec getResourceByName()

6

# QCM

**Q11.** Pour référencer une ressource string dans un fichier XML, on écrit :

a) @layout/...

b) @color/...

c) @string/...

d) @text/...

**Q12.** Quelle est la bonne manière d'utiliser une couleur définie dans colors.xml ?

a) @color/nom_couleur

b) @string/nom_couleur

c) #R.color.nom_couleur

d) R.color.nom_couleur

7

# QCM

**Q13.** L'intérêt principal de stocker les textes dans strings.xml est :

a) Réduire la taille du fichier APK

b) Faciliter la traduction et la maintenance

c) Améliorer les performances du layout

d) Rendre le code plus rapide

**Q14.** Quelle ressource permet d'adapter automatiquement la taille de la police selon la résolution ?

a) dimens.xml

b) colors.xml

c) strings.xml

d) styles.xml

8

# QCM

**Q15.** Lorsqu'on écrit android:text="@string/app_name", cela signifie :

a) On crée un texte directement dans le code

b) On fait référence à la chaîne app_name définie dans strings.xml

c) On définit la couleur du texte

d) On affiche le nom du package

**Q16.** Quelle bonne pratique est recommandée pour la gestion des textes ?

a) Écrire les textes directement dans les fichiers .xml de layout

b) Centraliser les textes dans strings.xml

c) Les déclarer dans le code Kotlin

d) Les stocker dans un fichier texte externe

9

# QCM

**Q17.** Qu'est-ce qu'une Activity dans Android ?

a) Une classe permettant de gérer les données locales

b) Une interface graphique représentant un écran de l'application

c) Un service exécuté en arrière-plan

d) Un fichier de configuration système

**Q18.** Quelle méthode du cycle de vie est appelée lorsque l'activité est créée pour la première fois ?

a) onStart()

b) onCreate()

c) onResume()

d) onPause()

10

# QCM

**Q19.** Quelle méthode est appelée **juste avant que l'activité ne devienne visible à l'utilisateur** ?

a) onPause()

b) onStop()

c) onStart()

d) onDestroy()

**Q20.** Quelle est la méthode appelée **lorsque l'utilisateur quitte temporairement l'activité (par exemple, un appel entrant)** ?

a) onPause()

b) onResume()

c) onStop()

d) onRestart()

11

# QCM

**Q21.** La pile d'activités (**Activity Stack**) fonctionne selon le principe :

a) FIFO (First In, First Out)

b) LIFO (Last In, First Out)

c) Ordre alphabétique

d) Aléatoire

**Q22.** Lorsque l'utilisateur appuie sur le bouton **Retour,** que se passe-t-il ?

a) L'activité actuelle est mise en pause et reste en mémoire

b) L'activité actuelle est terminée et retirée de la pile

c) L'application redémarre

d) Rien ne se passe

12

# QCM

**Q23.** Quelle méthode est utilisée pour sauvegarder l'état temporaire d'une activité (par exemple, le contenu d'un champ texte) ?
a) onSaveInstanceState()
b) onPause()
c) onStop()
d) onDestroy()

**Q24.** Quelle méthode permet de **restaurer l'état sauvegardé** après une rotation d'écran ?
a) onRestart()
b) onRestoreInstanceState()
c) onStart()
d) onResume()

13

# QCM

**Q25.** Une ***View*** en Android représente :
a) Un bouton, une image ou tout élément visible à l'écran
b) Un fichier XML de configuration
c) Une classe d'arrière-plan
d) Une ressource de texte

**Q26.** Quelle méthode est utilisée pour récupérer une ***View*** définie dans un fichier XML ?
a) getViewByName()
b) findViewById()
c) getElementById()
d) findByTag()

14

# QCM

**Q27.** Pour modifier dynamiquement le texte d'un TextView, on utilise :

a) setText()

b) setName()

c) getText()

d) updateView()

**Q28.** Pour gérer le clic sur un bouton, on peut définir un :

a) ToastListener

b) OnClickListener

c) ViewHandler

d) ClickAdapter

15

# QCM

**Q29.** Quel layout dispose ses éléments **en ligne horizontale ou verticale** ?

a) FrameLayout

b) LinearLayout

c) RelativeLayout

d) GridLayout

**Q30.** Dans un RelativeLayout, la position d'un élément est :

a) Fixe dans la fenêtre

b) Définie par rapport à d'autres éléments

c) Toujours centrée

d) Déterminée automatiquement

16

# QCM

**Q31.** Le ConstraintLayout permet :
a) De placer les éléments de manière flexible en définissant des contraintes entre eux
b) De créer uniquement des grilles
c) D'aligner automatiquement tous les boutons à gauche
d) De gérer les événements utilisateur

**Q32.** Dans un ConstraintLayout, une *contrainte* relie :
a) Deux activités entre elles
b) Une View à une autre View ou à son parent
c) Deux fichiers XML
d) Un View à une ressource

17

# QCM

**Q33.** Un *Intent* en Android sert principalement à :
a) Définir l'interface graphique d'une activité.
b) Envoyer un message entre composants Android.
c) Sauvegarder les préférences utilisateur.
d) Charger les fichiers de ressources.

**Q34.** Lequel des éléments suivants n'est **pas un type d'Intent** ?
a) Explicite
b) Implicite.
c) Diffusé (Broadcast)
d) Persistant

18

# QCM

**Q35.** Un Intent **explicite** est utilisé lorsque :

a) On connaît la classe cible de l'activité à lancer.

b) On veut laisser Android choisir la meilleure application.

c) On veut envoyer un message à plusieurs applications.

d) On veut lancer un service système.

19

# Questions du cours

2. Citez les **étapes nécessaires pour activer et utiliser View Binding** dans une activité Android.

a. **Activer View Binding** dans le fichier `build.gradle` du module :

```
buildFeatures {
        viewBinding true
    }
```

a. **Synchroniser le projet** avec Gradle.

b. **Déclarer une variable de binding** dans l'activité :

```
private lateinit var binding: ActivityMainBinding
```

a. **Initialiser le binding** dans `onCreate` :

```
binding = ActivityMainBinding.inflate(layoutInflater)
setContentView(binding.root)
```

a. **Accéder aux vues** via `binding.nomVue` au lieu de `findViewById`.

20

## Questions du cours

3. Donnez 3 exemples de ressources Android et leur rôle.
4. Expliquez le rôle d'un Listener en Android.
5. Donnez deux exemples de listeners et précisez leur utilisation.

21

## Exercice 1

- Complétez le code pour afficher dans un Toast le bouton sélectionné d'un RadioGroup.

```
val radioGroup = findViewById<RadioGroup>(R.id.radioGroup)
radioGroup.setOnCheckedChangeListener { group, checkedId ->
    val radioButton = findViewById<RadioButton>(checkedId)
    Toast.makeText(this, "Choix : ${radioButton.text}",
Toast.LENGTH_SHORT).show()
}
```

22